

Expert Routing & Top- k Gating.

A learned router scores every expert for every token, a softmax turns those scores into a distribution, and top- k selection keeps only the k strongest. The result is a layer whose *capacity* grows with the number of experts E but whose *compute* grows only with k . We derive the gate, both renormalisation conventions, the FLOP/parameter split, and noisy gating — and every number on the following pages is computed in full, nothing summarised or deferred.

THE EQUATION

$$y = \sum_{i \in \text{Top}_k(g)} \frac{p_i}{\sum_{j \in \text{Top}_k(g)} p_j} \text{Expert}_i(x), \quad p = \text{softmax}(g), \quad g = xW_g$$

Worked example. One token over $E=8$ experts (full softmax, top-2, renormalised gate); then a batch of 6 tokens $\times$ 8 experts as a coloured routing heatmap; then Mixtral's active-vs-total parameter arithmetic.

Topic 1 of 2 in this module · companion to the course page · v1.0

Contents

1	The router, the softmax, and sparsity	2
2	The routing pipeline (TikZ)	3
3	Worked example: one token, $E = 8$, top-2	3
4	A batch: the routing-probability heatmap	4
5	FLOPs and parameters: capacity vs. compute	4
6	Noisy top- k gating	5
7	Where this sits in the track	5
A	Reproducible (NumPy)	6

1 The router, the softmax, and sparsity

A Mixture-of-Experts (MoE) layer replaces a single feed-forward block with E independent experts $\text{Expert}_1, \dots, \text{Expert}_E$, each a full FFN $\mathbb{R}^d \rightarrow \mathbb{R}^d$. A tiny linear *router* (or *gate*) $W_g \in \mathbb{R}^{d \times E}$ scores the experts for a token $x \in \mathbb{R}^d$:

$$g = xW_g \in \mathbb{R}^E \quad (\text{gate logits}), \quad p = \text{softmax}(g) \in \mathbb{R}^E \quad (\text{routing probabilities}).$$

Dense MoE would run all E experts and combine them with weights p . *Sparse* MoE instead keeps only the k largest entries of g — the set $\text{Top}_k(g)$ — renormalises the gate over those k , and evaluates only those experts. The other $E - k$ experts are never touched.

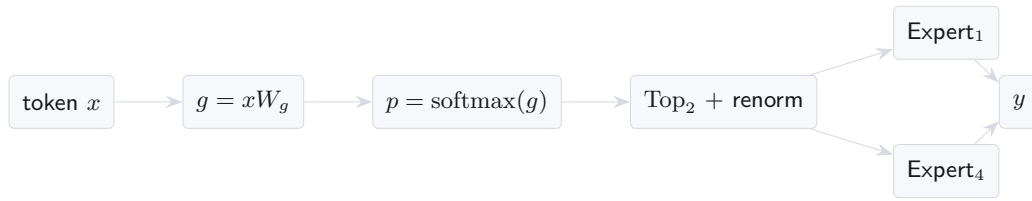
$$g = xW_g, \quad p = \text{softmax}(g), \quad y = \sum_{i \in \text{Top}_k(g)} \underbrace{\frac{p_i}{\sum_{j \in \text{Top}_k(g)} p_j}}_{w_i} \text{Expert}_i(x) \quad (1)$$

The weights w_i are the softmax probabilities *renormalised over the selected experts*, so $\sum_{i \in \text{Top}_k} w_i = 1$ and the output is a convex combination of the chosen experts.

Symbol	Meaning	Shape
x	token embedding	$\mathbb{R}^d$
W_g	router (gate) weights	$\mathbb{R}^{d \times E}$
g	gate logits	$\mathbb{R}^E$
$p = \text{softmax}(g)$	routing probabilities	$\mathbb{R}^E$
E, k	#experts, #selected (top- k)	scalar
$\text{Top}_k(g)$	indices of the k largest logits	set, $ \cdot = k$
w_i	renormalised gate weight for expert i	scalar
Expert_i	i -th feed-forward network	$\mathbb{R}^d \rightarrow \mathbb{R}^d$

The router is a one-layer classifier *over experts*. “Sparse” means each token wakes up only k of the E specialists. The model can store the knowledge of all E experts, but for any single token it pays the FLOPs of only k . Capacity scales with E ; compute scales with k . That decoupling is the entire reason MoE exists.

2 The routing pipeline (TikZ)



The dashed-out experts 2, 3, 5, 6, 7, 8 are not drawn because they are not evaluated for this token.

3 Worked example: one token, $E = 8$, top-2

Take a single token with router logits

$$g = (3.2, 1.1, 0.4, 2.5, -0.7, 1.8, 0.9, -1.3).$$

Step 1 — full softmax. Exponentiating (after subtracting $\max g = 3.2$ for stability) and normalising by $\sum_i e^{g_i - 3.2} = 2.0584$ gives

$$p = (0.4859, 0.0595, 0.0295, 0.2413, 0.0098, 0.1198, 0.0487, 0.0054), \quad \sum_i p_i = 1.$$

Step 2 — top-2 selection. The two largest entries are $p_1 = 0.4859$ (expert 1) and $p_4 = 0.2413$ (expert 4), so $\text{Top}_2(g) = \{1, 4\}$. **Step 3 — renormalise over the kept experts.** Their mass is $p_1 + p_4 = 0.7272$, so

$$w_1 = \frac{0.4859}{0.7272} = 0.6682, \quad w_4 = \frac{0.2413}{0.7272} = 0.3318, \quad w_1 + w_4 = 1.$$

Equivalently, take a softmax over just the selected logits $(g_1, g_4) = (3.2, 2.5)$:

$$w_1 = \frac{e^{3.2}}{e^{3.2} + e^{2.5}} = \frac{24.53}{24.53 + 12.18} = 0.6682, \quad w_4 = 0.3318,$$

which is identical — renormalising the probabilities and softmaxing the kept logits give the same numbers (dividing by the missing mass cancels). The layer output is

$$y = 0.6682 \cdot \text{Expert}_1(x) + 0.3318 \cdot \text{Expert}_4(x)$$

and experts 2, 3, 5, 6, 7, 8 are never run for this token.

Two conventions. *Renormalised* gating (Switch / Mixtral) divides by the kept mass so the weights sum to 1; this is Eq. (1). The alternative *raw-softmax* convention keeps the original p_i as weights without renormalising, giving $y = \sum_{i \in \text{Top}_k} p_i \text{Expert}_i(x)$ with $\sum w_i = p_1 + p_4 = 0.7272 < 1$. Renormalised weights make y a true convex combination and decouple its scale from how confident the router was; the raw form lets the router down-weight the whole block when it is unsure. Both appear in the literature — always check which one a paper uses.

4 A batch: the routing-probability heatmap

Now stack $N = 6$ tokens (rows) against the $E = 8$ experts (columns). Row-wise softmax of the logit matrix (see Appendix) gives the routing matrix $p = \text{softmax}(g)$. Cell intensity scales with the probability (capped at indigo!80); the two **bold** cells per row are that token's top-2 picks.

	e1	e2	e3	e4	e5	e6	e7	e8
t1	.486	.060	.030	.241	.010	.120	.049	.005
t2	.036	.325	.049	.030	.439	.040	.016	.066
t3	.021	.051	.558	.041	.031	.251	.037	.011
t4	.299	.045	.037	.403	.015	.067	.055	.081
t5	.051	.306	.046	.056	.068	.037	.413	.023
t6	.059	.040	.239	.036	.048	.053	.044	.481

Reading off the bold cells, the top-2 dispatch is

$$t1 \rightarrow \{1, 4\}, t2 \rightarrow \{5, 2\}, t3 \rightarrow \{3, 6\}, t4 \rightarrow \{4, 1\}, t5 \rightarrow \{7, 2\}, t6 \rightarrow \{8, 3\}.$$

Each row activates exactly 2 of 8 experts, so $6 \times 8 = 48$ possible (token, expert) pairs collapse to $6 \times 2 = 12$ that are actually computed — a $4\times$ reduction in expert FLOPs.

5 FLOPs and parameters: capacity vs. compute

Let an expert FFN cost F FLOPs and hold π parameters. A *dense* layer running all experts costs EF FLOPs per token; sparse top- k costs kF plus the negligible router cost dE :

$$\frac{\text{active FLOPs}}{\text{dense FLOPs}} = \frac{kF}{EF} = \frac{k}{E}.$$

For Mixtral-style $k=2$, $E=8$ this is $2/8 = 25\%$ of the expert FLOPs. Meanwhile the *parameters* stored are $E\pi$ (all experts live in memory) but only $k\pi$ are touched per token:

$$\text{total expert params} = E\pi, \quad \text{active expert params} = k\pi, \quad \frac{\text{active}}{\text{total}} = \frac{k}{E}.$$

MoE breaks the usual “more parameters $\Rightarrow$ more compute” coupling. Parameters scale with E (memory, knowledge capacity); per-token FLOPs scale with k (latency, cost). You buy a much larger model for the inference price of a small one — the only new cost is the memory to hold all E experts.

5.1 Mixtral 8 $\times$ 7B arithmetic

Mixtral 8 $\times$ 7B has $E=8$ experts and routes top- $k=2$. Only $k/E = 25\%$ of expert FLOPs fire. But because attention, embeddings, and norms are *shared* (not replicated per expert), the active parameter count is not simply $2/8$ of the total. The published figures are $\sim 47\text{B}$ total and $\sim 13\text{B}$ active per token:

$$\frac{\text{active}}{\text{total}} = \frac{13\text{B}}{47\text{B}} = 0.2766 = \mathbf{27.7\%}.$$

So Mixtral carries the knowledge capacity of a 47B model at roughly the inference cost of a 13B one (the active fraction 27.7% slightly exceeds the 25% FLOP fraction precisely because the shared non-expert weights are always active).

6 Noisy top- k gating

Pure top- k is deterministic, which can lock the router into picking the same experts early in training. The original Shazeer et al. gate adds tunable Gaussian noise *before* the top- k selection to encourage exploration:

$$\tilde{g} = xW_g + \varepsilon \odot \text{softplus}(xW_{\text{noise}}), \quad \varepsilon \sim \mathcal{N}(0, I), \quad p = \text{softmax}(\text{Top}_k(\tilde{g})) \quad (2)$$

A second learned matrix $W_{\text{noise}} \in \mathbb{R}^{d \times E}$ sets a *per-expert* noise scale (passed through softplus to stay positive). The noisy logits $\tilde{g}$ are used *only* for the top- k selection and the gate softmax; at inference the noise is dropped ($\varepsilon = 0$), recovering deterministic routing. Exploration early in training lets initially-unlucky experts still receive tokens and learn.

Top- k selection is non-differentiable: choosing “which k experts” is a discrete argmax with zero gradient almost everywhere. Gradients flow only through (a) the *gate weights* w_i of the chosen experts and (b) the *parameters* of those chosen experts. An expert that is never selected receives *no gradient at all* and never improves — which is the seed of expert collapse. This is why MoE needs an explicit load-balancing term (Topic 2) and why noisy gating exists: neither can be supplied by the selection’s own gradient, because there isn’t one.

7 Where this sits in the track

Back	Module 3.2 — the Mixtral overview (sparse MoE in one architecture).
This	Module 7.1 T1 — routing, top- k gating, both conventions, FLOP/param split, noisy gating.
Next	Module 7.1 T2 — load balancing, expert capacity, and avoiding collapse.

A Reproducible (NumPy)

Inputs: single-token logits g ; the 6×8 batch logit matrix; $k=2$. Verified outputs: single-token p as listed, top-2 = {1,4}, gate weights (0.6682,0.3318); batch dispatch as listed; $k/E = 0.25$; Mixtral active = $13/47 = 0.2766$.

```
import numpy as np
g1 = np.array([3.2,1.1,0.4,2.5,-0.7,1.8,0.9,-1.3])
p1 = np.exp(g1 - g1.max()); p1 /= p1.sum()
top = np.argsort(-p1)[:2]          # -> [0 3]  (experts 1,4)
w   = p1[top] / p1[top].sum()      # -> [0.6682 0.3318]  (renormalised)
print(np.round(p1,4)); print(top+1, np.round(w,4))

g = np.array([
  [3.2,1.1,0.4,2.5,-0.7,1.8,0.9,-1.3],
  [0.4,2.6,0.7,0.2, 2.9,0.5,-0.4,1.0],
  [-0.3,0.6,3.0,0.4,0.1,2.2,0.3,-0.9],
  [2.4,0.5,0.3,2.7,-0.6,0.9,0.7,1.1],
  [0.5,2.3,0.4,0.6,0.8,0.2,2.6,-0.3],
  [0.7,0.3,2.1,0.2,0.5,0.6,0.4,2.8]])
P = np.exp(g - g.max(1,keepdims=True)); P /= P.sum(1,keepdims=True)
print(np.argsort(-P,1)[:,:2] + 1)      # top-2 expert id per token
print(2/8, 13/47)                      # 0.25  0.27659...
```